

CS5013 Project Proposal — revision 2

Exchange Student Guide & Community Platform

Mikhail Novikov, Roll no. GE26Z858

Abdirakhim Ismailov, Roll no. GE26Z860

9 September 2026 (revision 1: 18 August 2026)

Why this revision exists. The course returned feedback on 28 August 2026: a portal holding “only a few bits of information” is not sufficient, and the project should be a wikipedia-style portal where information can be *added, edited and removed based on the consent of the community*. Approval was made conditional on our describing the changes, by 1 September. We did not reply by that date, and this document is that reply, late. Sections **1.1, 2, 4, 5, 7, 8 and 9** have changed; the feedback itself and our covering reply are in Appendix B.

1 Team information

Team ID: team-EXCHANGE.

- Member 1: Mikhail Novikov — roll no. GE26Z858 — department CSE — GitHub handle `rikire`
- Member 2: Abdirakhim Ismailov — roll no. GE26Z860 — department CSE — GitHub handle `abdra04-gif`

Primary contact email: `ge26z860@smail.iitm.ac.in`.

1.1 Module ownership — changed

Revision 1 split the work **Backend (Mikhail)** / **Frontend (Abdirakhim)**. That split is withdrawn. A horizontal split means every feature needs both people before it works, and neither owns a working thing end to end — which is precisely what the individual viva asks each of us to demonstrate.

The application is instead a **vertical-slice monolith**. A slice is one feature, self-contained: its controller, its service, its domain logic, its own repository, its own templates and its own tests. The ten slices are:

```
home articleview search taxonomy contribute moderate report media
wikilink backup
```

plus a small **shared** module (JPA entities and migrations, page layout, security) that changes only by agreement between the two of us.

A slice is therefore both the unit of work one person picks up and the unit ownership is measured against. Ownership is **claimed** in the repository when a slice is started and **measured** from git history afterwards, rather than assigned on paper today and contradicted by the log later. By the mid-demo the slice counts must be roughly even; if they are not, we say so and rebalance before the final phase. Spring Modulith enforces the boundary in a test, so a slice is a real compilation unit, not a label.

2 Problem statement — revised

Incoming exchange students at IIT Madras face a large volume of scattered practical information: FRRO registration, hostel check-in, SIM cards, bank accounts, food, transport and campus facilities. It is spread across office presentations, the institute website, and WhatsApp groups, and it goes out of date quietly.

Revision 1 proposed a centralised knowledge base curated by us. The course’s feedback is that this is too thin, and we agree: a guide written once by two students ages exactly as badly as the material it replaces. The product is therefore a **community-maintained knowledge base**: the students who have just solved a problem are the people who can describe its current state, and the Office of Global Engagement (OGE) keeps editorial control by approving what is published.

Anyone — with no account — can write a new article or propose an edit to an existing one. Every submission enters a moderation queue owned by OGE and becomes visible only when a moderator approves it. A moderator can also correct or remove a published article. That is the “consent of the community” loop: the community writes, OGE consents.

3 Users and stakeholder

Named stakeholder: Mr. Thukaram M Damodhar, Lead — International Academic Programs, Office of Global Engagement, IIT Madras. We approached him on 14 August 2026; he agreed to act as stakeholder and confirmed a centralised resource would be useful to his office and to incoming students. His acknowledgement is Appendix A.

Three roles drive the design, and each has its own journey written up in the repository:

- **Reader** — an exchange student with a question, arriving by search, by tag, from the landing page, or by following a link from another article.
- **Contributor** — anyone who writes an article or proposes an edit. No account, no login.
- **Moderator** — OGE staff, who approve, reject, correct, remove and pin.

Outstanding: the scoping meeting Mr. Thukaram offered has not yet happened. We have deliberately deferred it until there is a working system to react to rather than a description. Until then the acceptance criteria in §7 are ours, not his — stated here rather than presented as his sign-off.

4 Prior work and why it does not suffice — revised

Revision 1 compared the idea against what the stakeholder has today. Now that the product is a wiki, the more serious question is why not adopt an existing wiki engine, so that comparison is added.

1. **MediaWiki** — <https://www.mediawiki.org/>. The closest existing thing: a wikipedia-style engine with anonymous editing, links between articles and full history. *Why not:* publishing-after-approval is not core behaviour — it needs an extension ([FlaggedRevs](#) or [Extension:Moderation](#)), and OGE would then own a PHP install, its extensions, its upgrades and its spam defence. A two-person office needs a queue with an approve button, not a wiki farm. The course also requires us to build in Java, so adopting it is not open to us in any case.
2. **Confluence / Notion** — <https://www.atlassian.com/software/confluence>, <https://www.notion.so/>. Both are per-seat products designed as *internal* wikis; public pages are served from vendor domains, and a custom domain is a paid add-on. Cost rises with the

number of contributors, which is the wrong shape for “anyone may contribute”. *Not verified:* whether either permits contribution with no account at all — the sources we checked do not say, so we do not claim it either way.

3. **OGE orientation presentations** — internal PDF/PPT. Basic, one-directional, superficial for practical needs, and refreshed once a year.
4. **IIT Madras official website** — <https://www.iitm.ac.in/>. Authoritative on regulations, but scattered across sections and frequently out of date on practical matters.
5. **WhatsApp groups for international students**. Where the real answers currently live, and where they are lost: unstructured, unsearchable, the same questions repeat every intake.
6. **Travel blogs and city websites** (e.g. TripAdvisor). Not IITM-specific, silent on campus, and often stale.

5 Tech stack — revised

Language: **Java 21** (course policy). Versions below are the ones pinned in the build, verified against Maven Central on 2 September 2026.

- Build: Maven (wrapper-pinned), multi-module — the application and a small internal tooling module.
- Framework: Spring Boot 3.5.16 with **Spring Modulith 1.4.13**, which enforces the slice boundary of §1.1 as a test.
- Web: **server-rendered Thymeleaf** with Bootstrap 5. Revision 1 said “REST API”; withdrawn — there is no separate client to serve, and server-side rendering removes a whole layer for a two-person team.
- Database: H2 in development, PostgreSQL in production, schema under Flyway migrations.
- Search: Hibernate Search 7.2.6 with Lucene, full-text over article titles and bodies.
- Tests: JUnit 5, MockMvc, ArchUnit. Deployment: Docker Compose.
- Dropped from revision 1: Lombok (unused), and the REST/JSON layer above.

6 AI-usage statement

We use AI coding assistants during this project, in line with the course policy on AI-assisted programming. Every line of code we submit is code we understand and can explain in the viva, whether we typed it or the assistant generated it. In addition, this project keeps an automatic **prompt journal**: every prompt, the outcome, and the checks run are appended to the repository by a hook, so the record of how the code was produced is itself an artefact rather than a claim.

7 Verification plan — revised

Fixtures.

1. **Seed content.** Articles on the topics OGE named: FRRO registration, hostel check-in, SIM cards, bank accounts, food and prices, useful places. At least 10 by the design document and 20–30 by the mid-demo — enough that search and navigation are tested against real text rather than lorem ipsum.
2. **Corner cases.** Empty and single-character search queries, SQL-injection attempts, HTML and script tags inside article bodies, duplicate titles differing only in case, very long titles, wiki links to non-existent articles, oversized and disallowed uploads.
3. **Performance.** 100 articles of roughly 500 words; search must answer in under 2 seconds.

Acceptance criteria — revised for moderation. Revision 1 said a submitted article “appears immediately”. Under the moderation loop it does not, and the criteria change accordingly:

- Mr. Thukaram opens the application URL and finds an article by searching “FRRO registration”.
- A contributor submits “How to get a library card” with no account; it does *not* appear publicly, and it does appear in the moderation queue.
- Mr. Thukaram approves it in the queue, and it becomes visible on the site.
- He rejects a second submission, and it never becomes visible.
- He edits a published article directly, and the change is live without a queue step.
- He removes a published article; it stops resolving, and links to it turn red.

8 Milestone plan — revised

Revision 1 promised, by the design document: “database schema designed, Spring Boot project scaffolded with JPA entities and CRUD operations, and prototype homepage with static categories ready.” **That has not been met**, and it is more useful to say so than to restate it.

What exists on 9 September: the build, the CI gates, the process tooling, and the requirements layer — 26 functional requirements, 6 non-functional, 7 constraints, 25 use cases, one architecture decision record. What does not exist: any application code beyond a 51-line skeleton, no JPA, no migrations, no templates, no seed articles.

The cause is visible in the repository and is not a scheduling accident: effort went into process and specification ahead of product. The revision below is what follows from admitting it.

- **Design document, 11 Sep 2026.** Architecture note and slice map, one test named per module, this revised plan, honest risks. *Not* a working schema — that claim is withdrawn rather than quietly carried forward.
- **Skeleton, by 20 Sep 2026.** One vertical slice working end to end against a real database, Flyway migrations, and export/import. The first slice is the proof that the architecture is buildable; process tooling not needed for it is deferred.
- **Mid-demo, 9 Oct 2026.** The demo sentence, unchanged: a student opens the landing page, searches for “FRRO registration”, reads the article, follows a wiki link, proposes an edit with a photo — and the OGE moderator approves it and the change is live. The **must** features only; **should** and **could** wait.
- **Final, 6 Nov 2026.** Stakeholder-executed demo, hardening, deployment, handover documentation, viva preparation.

The scope lever is already in place: the 26 requirements are prioritised MoSCoW — 12 **must**, 5 **should**, 9 **could**. Cuts come off the bottom, and are recorded as constraints with a reason rather than dropped silently.

9 Risks and plan B — revised

1. **This project is not formally approved.** The course made approval conditional on a reply by 1 September; we did not send it. *Plan B*: this document is that reply, sent with it. If the direction is still judged wrong, the requirements are prioritised and the architecture is sliced, so scope can be cut to the 12 **must** features without redesigning anything.
2. **No application code exists with 30 days to the mid-demo.** The specification is far ahead of the product. *Signal that it is happening*: the first slice not finished by 20

- September. *Plan B*: demonstrate the **must** path only — read, search, submit, moderate — and drop media upload, reports, backlinks and export to after the mid-demo.
3. **An empty knowledge base at the demo.** A wiki with no articles demonstrates nothing, and writing them does not depend on the code. *Signal*: fewer than 10 drafts by 20 September. *Plan B*: both of us write content instead of features for two days; content is the one deliverable that cannot be produced on the last evening.
 4. **Two people working in parallel has never been tested.** All work so far has been joint, and the claim mechanism has never been used. *Signal*: the first two parallel slices collide in the same files. *Plan B*: serialise — one slice at a time, reviewed by the other — which costs speed but not correctness.

A Stakeholder acknowledgement email

Unchanged from revision 1: the email from Mr. Thukaram M Damodhar, Lead — International Academic Programs, Office of Global Engagement, IIT Madras (thukaram@ge.iitm.ac.in), received August 2026 in reply to our approach of 14 August 2026, confirming his willingness to act as stakeholder for this project. Reproduced verbatim in revision 1 of this proposal and in the project repository at `docs/stakeholder/acknowledgement.md`.

B Course scoping feedback, and our reply

The feedback of 28 August 2026 from Yuvraj Talukdar (CS23D009), copied to Prof. Omkar Dhawal, reproduced verbatim, together with our covering reply and a table mapping each part of the feedback to the requirement that answers it, is in the repository at `docs/course/scoping-feedback.md`.